Path 2: Cyber Security 101 > Module 4: Command Line > Room 3: Linux Shells

https://tryhackme.com/room/linuxshells

Linux Shells:
Learn about scripting and the different types of Linux shells.

>> Task 1: Introduction to Linux Shells

Suppose you are in a restaurant and have two options for your food. The first option is to order food from the menu, and the waiter will serve it. The second option is to cook your desired dish yourself in the kitchen. In terms of a Linux system, the kitchen here is the OS, and using the GUI of the OS is just like ordering the food from the menu, and the waiter will serve it for you. However, using the CLI means you would have to go to the kitchen (OS) and cook your desired food. In this example, Shell would help you cook your desired dish by giving you some recipe suggestions. Using CLI to perform operations in a Linux system gives you more power and control while carrying out the tasks.

Q1) Who is the facilitator between the user and the OS?
Answer: Shell

>> Task 2: How To Interact With a Shell?

Most Linux distributions use **Bash (Bourne Again Shell) as their default shell**. However, the default shell displayed when you open the terminal depends on your Linux distribution.

- Command: pwd (Print Working Directory)

By default, when you open a shell in most of the Linux distributions, you will be in your home directory. To see your current working directory, you can execute pwd, which stands for Print Working Directory.

- Command: cd (Change Directory)

However, you can change your directory as well. To do that, you can use cd (short for Change Directory).

- Command: ls

While using the GUI of an OS, you can see the contents of a directory on the screen. However, while using the shell, to see the contents of a directory, you must enter the ls command.

- Command: cat <file>

If you want to read the contents of a file, you can type the cat command in your shell.

- Command: grep <keyword/content> <file>

The grep command is a very popular command among Linux users. This powerful command can search for any word or pattern inside a file. Suppose you want to search for specific entries in a huge file. You can use the grep command along with the pattern of those entries, which will extract them for you. It also helps you to search for a specific keyword in a big file.

Q1) What is the default shell in most Linux distributions?
Answer: Bash

Q2) Which command utility is used to list down the contents of a directory?
Answer: ls

Q3) Which command utility can help you search for anything in a file?
Answer: grep

>> Task 3: Types of Linux Shells

Like the Command Prompt and PowerShell in Windows OS, Linux has different types of shells available, each with its own features and characteristics.

- Command: echo $SHELL

Multiple shells are installed in different Linux distributions. To see which shell you are using, type the command: echo $SHELL

- Command: cat /etc/shells

You can also list down the available shells in your Linux OS. The file "/etc/shells" contains all the installed shells on a Linux system. You can list down the available shells in your Linux OS by typing cat /etc/shells in the terminal.

- Example Command: zsh, sh, bash

To switch between these shells, you can type the shell name that is present on your OS, and it will open for you.

- Command: chsh -s /usr/bin/zsh

If you want to permanently change your default shell, you can use the command: chsh -s /usr/bin/zsh. This will make this shell as the default shell for your terminal.

- Bourne Again Shell:

Bourne Again Shell (Bash) is the default shell for most Linux distributions. When you open the terminal, bash is present for you to enter commands. Before bash, some shells like sh, ksh, and csh had different capabilities. Bash came as an enhanced replacement for these shells, borrowing capabilities from all of them. This means that it has many of the features of these old shells and some of its unique abilities. Some of the key features provided by bash are listed below:

- Bash is a widely used shell with scripting capabilities.

- It offers a tab completion feature, which means if you are in the middle of completing a command, you can press the tab key on your keyboard. It will automatically complete the command based on a possible match or give you multiple suggestions for completing it.

- Bash keeps a history file and logs all of your commands. You can use the up and down arrow keys to use the previous commands without typing them again. You can also type history to display all your previous commands.

- Friendly Interactive Shell:

Friendly Interactive Shell (**Fish**) is also not default in most Linux distributions. As its name suggests, it focuses more on user-friendliness than other shells. Some of the key features provided by fish are listed below:

- It offers a very simple syntax, which is feasible for beginner users.

- Unlike bash, it has auto spell correction for the commands you write.

- You can customize the command prompt with some cool themes using fish.

- The syntax highlighting feature of fish colors different parts of a command based on their roles, which can improve the readability of commands. It also helps us to spot errors with their unique colors.

- Fish also provides scripting, tab completion, and command "history" functionality like the shells mentioned in this task.

- Z Shell

Z Shell (**Zsh**) is not installed by default in most Linux distributions. It is considered a modern shell that combines the functionalities of some previous shells. Some of the key features provided by zsh are listed below:

- Zsh provides advanced tab completion and is also capable of writing scripts.

- Just like fish, it also provides auto spell correction for the commands.

- It offers extensive customization that may make it slower than other shells.

- It also provides tab completion, command history functionality, and several other features.

Features	Bash	Fish	Zsh
Full Name	The full form of Bash is Bourne Again Shell.	The full form of Fish is Friendly Interactive Shell.	The full form of Zsh is Z Shell.
Scripting	It offers widely compatible scripting with extensive documentation available.	It has limited scripting features as compared to the other two shells.	It offers an excellent level of scripting, combining the traditional capabilities of Bash shell with some extra features.
Tab Completion	It has a basic tab completion feature.	It offers advanced tab completion by giving suggestions based on your previous commands.	Its tab completion capability can be extended heavily by using plugins.
Customization	Basic level of customization.	It offers some good customization through interactive tools.	Advanced customization through oh-my-zsh framework.
User Friendliness	It is less user-friendly, but being a traditional and widely used shell, its users are quite familiar and comfortable with it.	It is the most user-friendly shell.	It can be highly user-friendly with proper customization.
Syntax Highlighting	The syntax highlighting feature is not available in this shell.	The syntax highlighting is built-in to this shell.	The syntax highlighting can be used with this shell by introducing some plugins.

Q1) Which shell comes with syntax highlighting as an out-of-the-box feature?
Answer: Fish

Q2) Which shell does not have auto spell correction?
Answer: Bash

Q3) Which command displays all the previously executed commands of the current session?
Answer: history

>> Task 4: Shell Scripting and Components

A shell script is nothing but a set of commands. Suppose a repetitive task requires you to enter multiple commands using a shell. Instead of entering them one after one on every repetition of that task, which may take more of your time, you can combine them into a script. To execute all those commands, you will only execute the script, and all the commands will be executed. All the shells mentioned in the previous tasks have scripting capabilities. Scripting helps us to automate tasks. Before learning how to write a script, we need to know that even though Linux shells have scripting capabilities, this does not mean that you can only make a script using a shell. Scripting can be done in various programming languages as well.

- Unlike the other commands we type in the shell, we first need to create a file using any text editor for the script. The file must be named with an extension **.sh**, the default extension for bash scripts.

Command: nano first_script.sh

Every script should start from shebang. Shebang is a combination of some characters that are added at the beginning of a script, starting with **#!** followed by the name of the interpreter to use while executing the script. As we are writing our script in bash, let's define it as the interpreter in the shebang.

#!/bin/bash (first_script.sh > **#!/bin/bash**)

We are all set to write our first script now. There are some fundamental building blocks of a script that together make an efficient script. Let's learn and utilize these script constructs to write one script ourselves.

- Variables:

A variable stores a value inside it. Suppose you need to use some complex values, like a URL, a file path, etc., several times in your script. Instead of memorizing and writing them repeatedly, you can store them in a variable and use the variable name wherever you need it.

The script below displays a string on the screen: "Hey, what's your name?" This is done by echo command. The second line of the script contains the code **read name**. **read** is used to take input from the user, and **name** is the variable in which the input would be stored. The last line uses **echo** to display the welcome line for the user, along with its name stored in the variable.

```
# Defining the Interpreter
#!/bin/bash
echo "Hey, what's your name?"
read name
echo "Welcome, $name"
```

Now, save the script by pressing **CTRL+X**. Confirm by pressing **Y** and then **ENTER**.

To execute the script, we first need to make sure that the script has execution permissions. To give these permissions to the script, we can type the following command in our terminal: chmod +x first_script.sh

Now that the script has execution permissions use **./** before the script name to execute it. We use **./** before the script to run rather than typing the script name directly because **./** tells the shell to execute the file that is present in the current directory. If you don't define **./** before the script name, the shell will search the script in the PATH environment variable (that contains all the directories except the current one), and it will not find the defined script in any of those directories and generate an error. The below terminal shows the script in which we utilized the variables:

```
user@ubuntu:~$ ./first_script.sh
Hey, What's your name?
cybersantosh
Welcome, cybersantosh
```

- Loops:

Loop, as the name suggests, is something that is repeating. For example, you have a list of many friends, and you want to send them the same message. Instead of sending them individually, you can make a loop in your script, give your friend list to the loop and the message, and it will send that message to all your friends.

For a general explanation of loops, let's write a loop that will display all numbers starting from 1 to 10 on the screen. First, create a new file named **loop_script.sh**, then enter the code below. Save your file by pressing **CRTL+X**, then confirm with **y** and then **ENTER**.

```
# Defining the Interpreter
#!/bin/bash
for i in {1..10};
do
echo $i
done
```

The first line has the variable **i** that will iterate from 1 to 10 and execute the below code every time. **do** indicates the start of the loop code, and **done** indicates the end. In between them, the code we want to execute in the loop is to be written. The "for" loop will take each number in the brackets and assign it to the variable **i** in each iteration. The **ceho $i** will display this variable's value every iteration.

./loop_script.sh (Script Execution)

When executed according to the script's logic, it would display the numbers from 1 to 10.

- Conditional Statements:

Conditional statements are an essential part of scripting. They help you execute a specific code only when a condition is satisfied; otherwise, you can execute another code. Suppose you want to make a script that shows the user a secret. However, you want it to be shown to only some users, only to the high-authority user. You will create a conditional statement that will first ask the user their name, and if that name matches the high authority user's name, it will display the secret.

First, create a new file named **conditional_script.sh**, then enter the code below. Save your file by pressing **CRTL+X**, then confirm with **y** and then **ENTER**.

```
# Defining the Interpreter
#!/bin/bash
echo "Please enter your name first:"
read name
if [ "$name" = "Stewart" ]; then
        echo "Welcome Stewart! Here is the secret: THM_Script"
else
        echo "Sorry! You are not authorized to access the secret."
fi
```

The above script takes the user's name as input and stores it into a variable (studied in the Variables section). The conditional statement starts with "if" and compares the value of that variable with the "string Stewart"; if it's a match, it will display the secret to the user, or else it will not. The "fi" is used to end the condition.

conditional_script.sh (Script Execution)

Following is the terminal showing the script execution when the user name matches the authorized one defined in the script.

```
:
user@tryhackme:~$ ./conditional_script.sh
Please enter your name first:
Stewart
Welcome, Stewart! Here is the secret: THM_Script
```

However, the following terminal shows the script execution when the user name does not match the authorized one defined in the script.

```
:
user@tryhackme:~$ ./conditional_script.sh
Please enter your name first:
Alex
Sorry! You are not authorized to access the secret.
```

- Comments:

Sometimes, the code can be very lengthy. In this scenario, the code might confuse you when you look at it after some time or share it with somebody. An easy way to resolve this problem is to use comments in different parts of the code. A comment is a sentence that we write in our code just for the sake of our understanding. It is written with a # sign followed by a space and the sentence you need to write. For example, let's rewrite the script we discussed in the conditional statements section and add comments to it.

Open the **conditional_script.sh** with nano, then add the comments starting with a **#** sign. Save your file by pressing **CRTL+X**, then confirm with **y** and then **ENTER**.

```
# Defining the Interpreter
#!/bin/bash

# Asking the user to enter a value.
echo "Please enter your name first:"
```

```
# Storing the user input value in a variable.
read name

# Checking if the name the user entered is equal to our required name.
if [ "$name" = "Stewart" ]; then

# If it equals the required name, the following line will be displayed.
echo "Welcome Stewart! Here is the secret: THM_Script"

# Defining the sentence to be displayed if the condition fails.
else
        echo "Sorry! You are not authorized to access the secret."
fi
```

See how easy a script looks with comments. Comments don't affect the working of any script. A good script always has some comments. The example shown above contains a comment for each line. This is just a better explanation of its concept. However, the best way to include comments is to define them in the major and complex areas of the script.

Note: Other types of variables, loops, and conditional statements can also be used to achieve different tasks. Moreover, multiple lines of comments can also be added within a single comment. However, it is not the scope of this room.

Q1) What is the shebang used in a Bash script?
Answer: #!/bin/bash

Q2) Which command gives executable permissions to a script?
Answer: chmod +x

Q3) Which scripting functionality helps us configure iterative tasks?
Answer: loops

>> Task 5: The Locker Script

In the previous task, we studied variables, loops, and conditional statements in shell scripting. Let's use that knowledge to create a shell script that utilizes all these components.

- Requirement:

A user has a locker in a bank. To secure the locker, we have to have a script in place that verifies the user before opening it. When executed, the script should ask the user for their name, company name, and PIN. If the user enters the following details, they should be allowed to enter, or else they should be denied access.

Username: John
Company name: Tryhackme
PIN: 7385

- Script:

```
# Defining the Interpreter
#!/bin/bash

# Defining the variables
username=""
companyname=""
pin=""

# Defining the loop
for i in {1..3}; do
# Defining the conditional statements
        if [ "$i" -eq 1 ]; then
                echo "Enter your Username:"
                read username
        elif [ "$i" -eq 2 ]; then
                echo "Enter your Company name:"
                read companyname
        else
                echo "Enter your PIN:"
                read pin
        fi
done

# Checking if the user entered the correct details
```

```
if [ "$username" = "John" ] && [ "$companyname" = "Tryhackme" ] && [ "$pin" =
"7385" ]; then
        echo "Authentication Successful. You can now access your locker, John."
else
        echo "Authentication Denied!!"
fi
```

Script Execution: **Executing the Locker Script**

```
user@tryhackme:~$ ./locker_script.sh
Enter your Username:
John
Enter your Company name:
Tryhackme
Enter your PIN:
1349
Authentication Denied!!
```

Q1) What would be the correct pin to authenticate in the locker script?
Answer: 7385

>> Task 6: Practical Exercise

We have placed a script on the default user directory **/home/user** of the attached Ubuntu machine. This script searches for a specific keyword in all the files (with .log extension) in a specific directory.

Note: Some changes are required inside the script file before you execute it. When you open the machine according to the instructions in task #2, you will be able to gain the session as a normal user. However, we recommend you to become the root user in order to search for the flag in all the files of the given directory. To become one, you only need to type the following command and enter the password of the user (The credentials are included in Task 2): username: user :: password: user@Tryhackme

Become Root User:
user@tryhackme:~$ sudo su

You can make the changes in the script file by keeping in view the following details:
Flag: thm-flag01-script
Directory: /var/log

Hint: Look for empty double quotes **" "** inside the script file and fill them. Make sure not to leave any space between them.

Q1) Which file has the keyword?
Answer: authentication.log
Hint: grep thm-flag01-script /var/log/*.log

Q2) Where is the cat sleeping?
Answer: under the table